Candidate Comparison and Local Constraints for Mathematical Answer Selection

DASE7506 MP2 · Selecting Reliable LLM Reasoning

Meng Weitao · 3036746155 Release 1.0

1. Research Question and Evaluation Design

Four candidate solutions to the same mathematics problem may each contain fluent calculations while interpreting quantities or the requested target differently. This study asks whether comparing candidate representations and answer groups, together with local constraints that apply only to explicitly supported syntactic patterns, can make answer selection more reliable than assessing candidates individually. The working hypothesis is that candidate comparison provides complementary evidence, whereas local arithmetic and quantity constraints can correct only a subset of errors. More complex ensembles do not necessarily improve generalization.

The task requires one candidate index from 0 to 3 for each problem. Supervision is determined by the course's fixed parser: a candidate is labeled correct if its final numerical answer equals the reference answer. A problem may therefore have multiple positive candidates. Official accuracy measures the correctness of the selected final answer, rather than the validity of every step in the selected reasoning. All experiments use the supplied problems, candidate texts, and representations of shape [N, 4, 1024], with no external training data or additional pretrained weights.

Split	Problems	Hard problems	Role in this study
Training	4,755	2,114	Model fitting and construction of out-of-fold training features
Validation	798	355	Method development, global parameter selection, and ablation
Test	467	467	Final scoring and reproduction of the frozen method

A hard problem is defined as one with at least two distinct valid parsed answers. The final method answers all 443 remaining validation problems correctly. Both full-validation and hard-validation accuracy are therefore reported so that easier problems do not obscure the ability to distinguish candidates. Every test problem meets the hard-problem definition, but this does not imply that the hard-validation and test distributions are identical.

The final method achieves 770/798 = 96.4912% on the full validation set and 327/355 = 92.1127% on hard validation. Its official test result is 395/467 = 84.58244111349036%. These results support an improvement on the development set, but do not establish a comparable generalization gain. Throughout this report, the official test score is the submission score, while hard-validation accuracy is explicitly treated as a development metric affected by model selection.

The experimental boundaries are as follows. Weights are fitted only on training data; validation data are used repeatedly to select architectures, features, and rule strengths. Test scores for earlier methods were already known, but subsequent optimization was not based on test-error analysis. The main method was frozen before its first final test evaluation. A mixed-currency edge case was subsequently fixed using synthetic review cases only, without tuning against the test set. Release 1.0 is a behavior-preserving reorganization of the reviewed version that had already been scored. It retains the algorithm, weights, and selected parameters; reruns serve only to reproduce existing results. This release is not a new independent generalization experiment. The

data, numerical parser, and scorer remain unchanged, and test reference answers are not used during training or inference.

2. Method: From Individual Evidence to Candidate Comparison

The method has three stages: shared prediction, answer-group comparison, and conservative constraints. Shared prediction supplies scores from models with different inductive biases. Answer-group comparison uses relationships among candidates for the same problem. Constraints adjust existing candidates only when supported structures are present; they do not generate new answers.

The input features comprise 84 basic features, 59 reasoning features, and 89 semantic features, covering formatting, explicit calculations, target quantities, and quantitative relationships. The seven shared components are two pairwise linear models, an HGB meta-model, a CatBoost pairwise ranker, a conditional network, a pairwise HGB model, and a model using quadratic features derived from PCA. HGB denotes histogram-based gradient boosting. Multiple final prediction heads share these components' computations and weights, avoiding the need to store a separate complete ensemble for each head.

Second-level learning uses three-fold out-of-fold predictions from the training set. The four candidates for a problem always remain in the same fold. Within the meta-model, an additional four-fold procedure generates the required base scores. When a base component predicts a problem, that problem is excluded from its training data. Second-level models are fitted on hard training problems. This reduces the stacking bias caused by directly using in-sample scores, but cannot eliminate the optimism introduced by repeated validation-set selection.

Candidates are first placed in a canonical order based on their text and representation content, and outputs are then restored to the input order. Candidates whose parsed numerical answers are equivalent under the official parser form an answer group. Scores within a group are averaged, preventing repeated copies of the same answer from receiving an arbitrary multiplicative weight. The 422-dimensional stable-comparison features are extended with 30 quantity-binding features and 6 format-transfer features, producing a 458-dimensional input for the binding head. The binding head and the existing compact ensemble are blended at a ratio of 1:4.

Key settings are as follows: the binding head uses 400 trees, 31 leaf nodes, a minimum of 20 samples per leaf, L2 regularization of 20, and a learning rate of 0.05; the stable head uses 200 trees and 15 leaf nodes; CatBoost uses depth 6 and 266 trees; and the conditional network uses 32 hidden units and 40 epochs. The out-of-fold split seed is 20261030, the main HGB seed is 20261108, the CatBoost seed is 20261022, and the PCA seed is 20261024. Exact settings are recorded in the training code and checkpoint configuration. Score comparisons use float64.

There are four categories of local constraints. First, when the top-ranked answer group conflicts with a unique final boxed answer and the target answer is already present among the candidates, 0.25 probability mass is transferred. Second, a penalty is applied when every piece of evidence for the same answer conflicts with the target unit. Penalty strengths are 2 for physical dimensions and 0.25 for the type of object being counted. Third, when there is a unique salient error in explicit arithmetic, a fractional right-hand side, and subsequent calculations, a restricted abstract syntax tree (AST) propagates the correction, with a transfer strength of 0.5. The rule abstains on unsupported algebra, unit conversions, and rounding cases.

Fourth, production-profit and inventory-profit templates check explicit resource-cost relationships and relationships involving a single purchase followed by partial sales, respectively. Both use a support strength of 2. Inventory net profit is $m \cdot s + (n-m) \cdot k \cdot b - n \cdot b$, where n and b denote the purchased quantity and unit purchase price, m and s denote the sold quantity and unit selling price, and k is the multiplier determining the remaining items' value relative to their purchase cost. These rules require the template to account for the explicit numbers, consistent currencies,

and a derived value that already appears among the candidates. The price-percentage rule has weight 0. Template matching is a check against a limited syntax, not a complete semantic proof.

3. Experimental Comparisons and Unsuccessful Approaches

The comparisons are organized by information source and modeling approach. Every row uses the same validation problems and official numerical parser. Counts of correct selections are reported to make small changes interpretable. The full method correctly selects 23 more hard-validation answers than the linear baseline. However, these methods differ in more than one factor; individual module contributions should be interpreted together with the ablations in the next section.

Method	Correct on hard validation	Hard-validation accuracy	Correct on full validation
Majority voting	245/355	69.01%	688/798
Best initial representation-only model	286/355	80.56%	729/798
Linear baseline: representations, features, and answer groups	304/355	85.63%	747/798
Early two-model ensemble	309/355	87.04%	752/798
Candidate-group comparison model	316/355	89.01%	759/798
Shared ensemble with format transfer	321/355	90.42%	764/798
Shared ensemble with the quantity-binding head	322/355	90.70%	765/798
Full method: local constraints added	327/355	92.11%	770/798

Majority voting is substantially weaker than the learned methods, indicating that answer frequency alone is insufficient to determine correctness. The gap between the representation-only model and the linear model with mixed features motivates further study of structural information. Candidate-group comparison and subsequent ensembling further improve development results. Nevertheless, these configurations underwent different amounts of search. The differences in this table cannot be attributed directly to any single feature, nor do they establish that the model has learned to verify complete reasoning chains.

Additional computation did not always pay off. Candidate deletion and duplication augmentation expanded 2,114 hard training problems into 8,456 groups. For each problem, the teacher excluded the original problem from its training, and the weights of variants derived from the same source summed to 1. This experiment took 492.83 seconds but did not exceed the then-current result of 321/355. A second-level search using PCA latent features took 340.85 seconds and likewise did not exceed 321/355. Ensemble calibration over six weight settings and six transformations took 5.52 seconds, also without improving its baseline. These results suggest that additional training variants or representations do not necessarily resolve weaknesses in understanding the roles of quantities.

The stable-head search compared 72 head configurations and 216 ensemble settings in a run lasting 404.85 seconds. Greedy selection of an additional binding head from the same library then took 47.53 seconds and added only one correct answer. A subsequent beam search took 42.68 seconds and selected the same result. Broad arithmetic penalties, general consistency penalties, and their pooled variants also failed to exceed their respective baselines. The development gains from restricted rules were concentrated in very few problems, making coverage and generalization checks essential.

These are records of individual runs on the local machine, some of which used caches. Their durations cannot be added together and treated as the total research cost. The search space for each run was written into code and records before execution, but the overall research process adapted to validation feedback and was not preregistered. Retained experimental records support inspection of these comparisons, while the released code focuses on training, prediction, and the necessary checks for the final method.

4. Ablation and Generalization: Strengths and Limits of the Evidence

Each fixed-configuration ablation removes one module while retaining the other parameters, without refitting or retuning. It measures that module's effect on the current development result. Because the validation set was already used for selection, these results are not independent causal validation, and the decreases in correct-answer counts cannot simply be added together.

Configuration	Correct on hard validation	Change from the full method	Correct on full validation
Full method	327/355	0	770/798
Without the quantity-binding head	326/355	-1	769/798
Without format transfer	326/355	-1	769/798
Without the target-unit constraint	325/355	-2	768/798
Without fractional-arithmetic repair	326/355	-1	769/798
Without the production-profit constraint	326/355	-1	769/798
Without the inventory-profit constraint	326/355	-1	769/798

The target-unit constraint affects two problems in this configuration; each other module affects one. Quantity binding, format, unit, and arithmetic signals may be complementary, but their marginal gains are small. These ablations also do not fully resolve the question of reasoning understanding versus surface style. Text features contain both formatting and semantic cues, and the present experiments cannot completely separate their effects.

Rule coverage further limits the conclusions. Format transfer identifies 15 source answer groups in the training set, with correct target answers in 14 cases. Restricted fractional-arithmetic repair identifies only two source groups, both with correct targets. The production-profit and inventory-profit rules have zero matches in the training set. These latter rules use general syntactic patterns designed after validation-error analysis and were checked with synthetic problems varying items, prices, and quantities. They contain no problem IDs or per-problem answer tables. However, synthetic checks establish implementation behavior only for the specified inputs; they do not supply evidence of reliability on the real training distribution.

The final test result is 395/467, which is 7.53 percentage points below the hard-validation result of 327/355. The historical linear baseline also achieved 395/467, while the early two-model ensemble achieved 400/467 = 85.65%; their original experiment labels were v1 and v3, respectively. Thus, continued improvement on the development set did not translate into a demonstrated test gain. The entire gap cannot be attributed conclusively to overfitting, but extensive validation-based selection and distribution differences are plausible explanations. The model previously selected using validation data was retained, rather than switching back to the early ensemble on the basis of known test scores.

Validation-error analysis indicates that quantity roles, omitted conditions, and target objects are harder to handle than isolated multiplication or division errors. Every calculation may be correct while the solution as a whole answers a different question. English lexical variation,

cross-sentence ellipsis, complex cost relationships, and rounding also fall beyond the templates' reliable coverage. The present evidence supports the claim that local rules can correct a small number of development errors. It does not support the claim that greater model complexity has produced stronger general reasoning ability. Further research should first establish an audit set that has not been used for selection, then evaluate simpler models, rule coverage, and confidence calibration.

5. Resource Constraints, Code Review, and Retraining Reproducibility

Release 1.0 replaces layered historical filenames and configuration directories with stable interfaces: code/submission.py for prediction, code/train.py for training, and code/evaluate.py for evaluation. Inference assets are code/checkpoint/model.joblib and code/checkpoint/config.json. This reorganization changes file structure without changing the model architecture, learned weights, or constraint strengths. Equivalence checks confirm identical selections and scores for the same inputs.

Item	Recorded result
All uncompressed inference assets	5,537,559 bytes = 5.281028 MiB
Complete test evaluation: fresh prediction process plus fresh scoring process	3.0640 seconds
Prediction process / scoring process	3.0142 / 0.0498 seconds
Refitting all weights from training data	134.69 seconds
Validation agreement after retraining	Identical selections on 798 problems; maximum score difference 0
Local environment	Apple M5, macOS 26.6.2, Python 3.12.14

Timing includes process startup, imports, input and weight loading, feature computation, all ensemble components, and separate scoring. BLAS is restricted to one thread, and OpenMP/CatBoost to at most four. Ensemble members and evaluation processes run sequentially. The asset count includes weights, learned transformations, and configuration files; inference does not depend on training caches or existing predictions. The course requires total assets of at most 16 MiB and complete four-thread evaluation within 30 seconds on the specified Xeon Platinum 8457C. Measurements on the local machine do not replace eligibility verification on that CPU.

Retraining reconstructs the seven shared components, three-fold base predictions, and all final prediction heads from the original training data, reusing only the selected configurations. Validation data are used solely for an agreement check after fitting is complete. This procedure neither evaluates the test set nor searches parameters again. Retraining outputs do not overwrite the released weights. Hashes of weights, configuration, and runtime source files are stored in code/release.json; official scoring, resource, and retraining records are stored in code/results/.

Code review focused on the interface for new problem groups and on rule boundaries. A previously identified issue allowed profit templates to accept different currency symbols without checking consistency. It was reproduced with synthetic cases and fixed by making the rules abstain on mixed currencies. No other models were changed, and no rules were added on the basis of test errors. The 32 checks in the reorganized release cover the four-candidate and representation-shape requirements, rejection of NaN/Inf, empty batches and single problems, input immutability, repeated calls, arbitrary IDs, all 24 candidate permutations, blocked reference-answer reads, and rule boundaries.

Original-data checks and source-file hash verification confirm that the fixed benchmark remains unchanged. Scores, candidate selections, full official evaluation, and training from scratch together provide reproducibility evidence. Passing tests guarantees only the covered behaviors, not semantic correctness for arbitrary natural language or bitwise equality across platforms. Dependency versions should match the recorded environment, and official evaluation should be rerun on other platforms.

6. Conclusions, Usage, and Research Disclosure

The main finding is that candidate comparison and a small set of constraints improve answer-selection accuracy on a repeatedly used development set, but this improvement is not confirmed on the test set. The final method achieves 92.11% on hard validation and 84.58% on the official test set. Shared computation keeps a method with multiple prediction components within a small asset footprint and a low local inference cost. Relative to the early ensemble, its additional complexity is not yet supported by an independent accuracy gain.

Release 1.0 is the only current formal code release and is a behavior-preserving reorganization of the reviewed method. The algorithm is retained to preserve the development choice made before test scoring, rather than to select the highest known test score. Version cleanup removes unnecessary implementation copies and intermediate artifacts while retaining the results, configurations, ablations, and provenance needed to explain the research conclusions. Future methodological work should establish a separate development and audit protocol instead of treating this test set as tuning feedback.

To reproduce the results, start with Python 3.12, install the locked dependencies from requirements.txt in code/, and run check_data.py to check the course data. Using the supplied checkpoint, run evaluate.py --split test --reproduce to obtain the official score and resource records including startup overhead; retraining is not required. Use train.py to refit all weights and evaluate.py --split validation --reproduce to reproduce validation results. See REPRODUCE.md for exact commands, output paths, and instructions for supplying a new audit dataset.

For new problem groups, the selector reads only the problem, four candidates, and representations with the course-specified dimensions. It returns candidate indices in input order. The number of problems and their IDs do not influence answer selection. The method uses no external service, live LLM generation, or reference-answer reads. The accuracy field in the official scoring JSON is 84.58244111349036; this is the value to use for submission.

OpenAI Codex contributed to model and feature design, coding, experiments, testing, review. Subagents contributed to requirements checks, some experiments, and review of the reorganization.

Sources and supporting evidence: guide/GUIDE.md, supplied with the assignment, for the task, scoring, and constraints; code/README.md, common.py, and the fixed scoring tools for interfaces and numerical parsing; the manifest, NOTICE, and LICENSE in code/data/ for provenance and licensing; and the retained scoring, ablation, search, resource, review, and retraining records in code/results/ for the experimental values reported here.